



CV 项目

人脸识别

课程内容

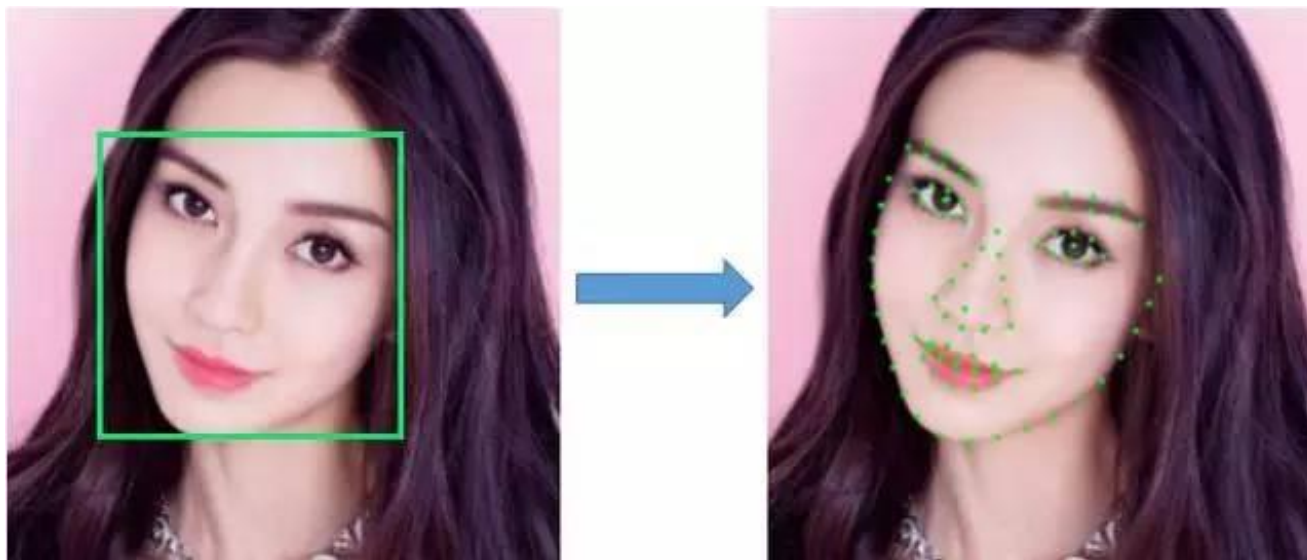
🕒 人脸识别概述

🕒 MT CNN

🕒 Face Net

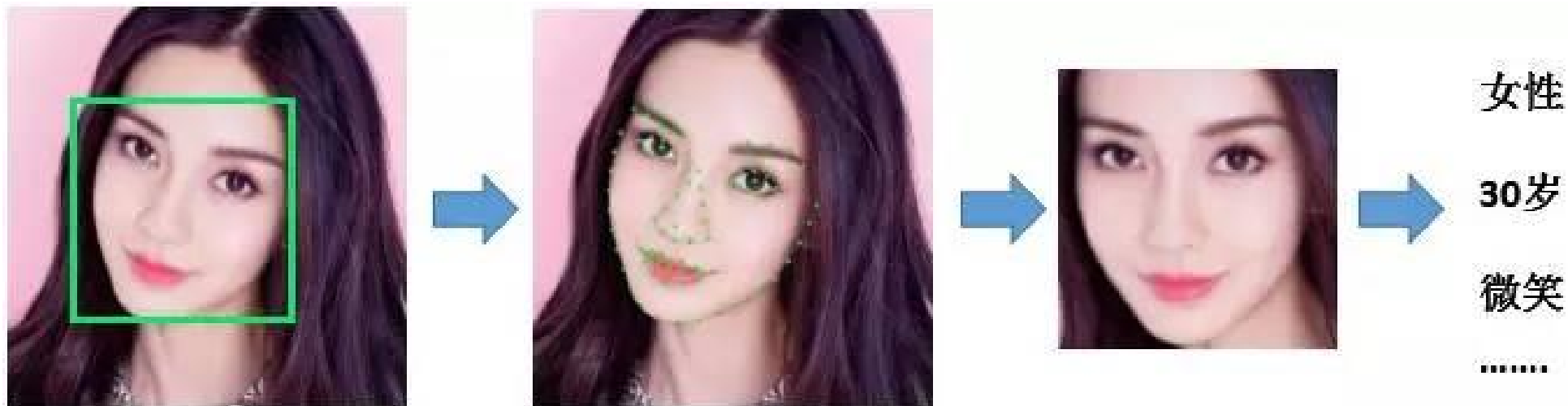
人脸识别概述

- ⌚ 人脸检测(Face Detection): 从待检测的图像中获取人脸所在位置的一项技术;
- ⌚ 人脸匹配(Face Alignment): 从定位出的人脸上五官关键点坐标的一项技术;



人脸识别概述

⌚ 人脸属性识别(Face Attribute): 是识别出人脸的性别、年龄、姿态、表情等属性值的一种技术。



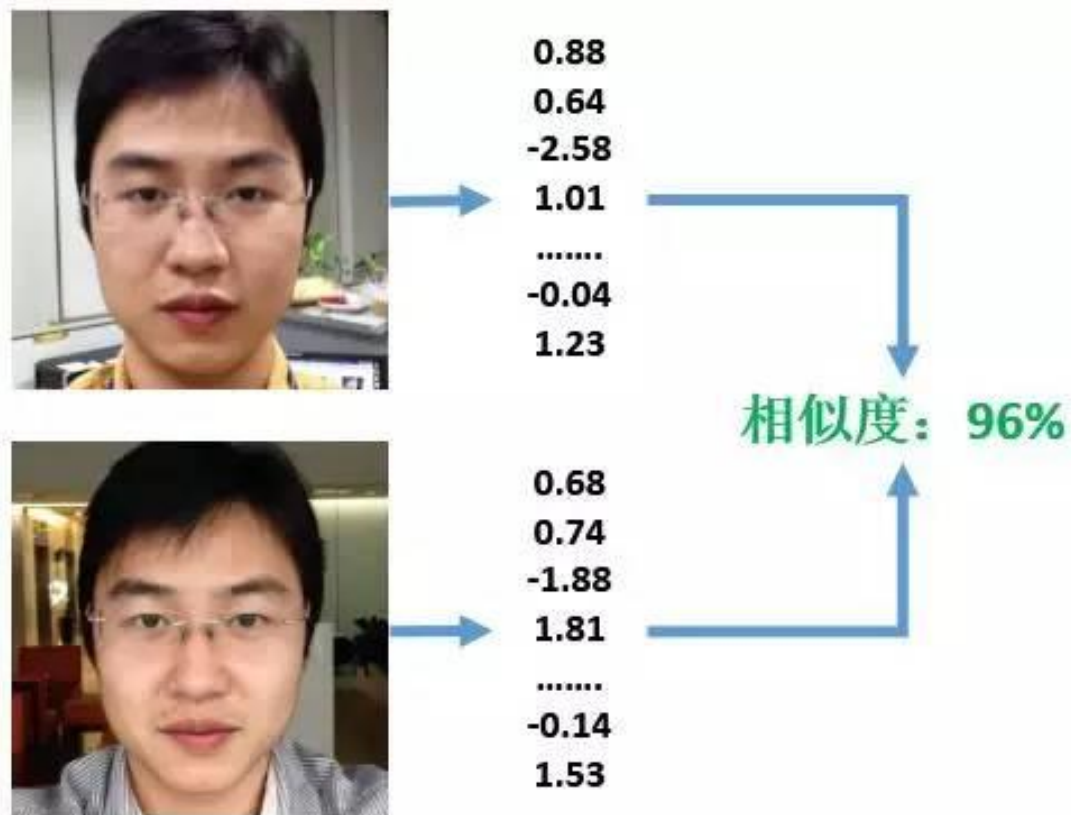
人脸识别概述

⌚ 人脸特征提取(Face Feature Extraction): 将一张人脸图像转换为固定维度的特征向量的过程, 这个向量具有表述这个人脸特征的能力。



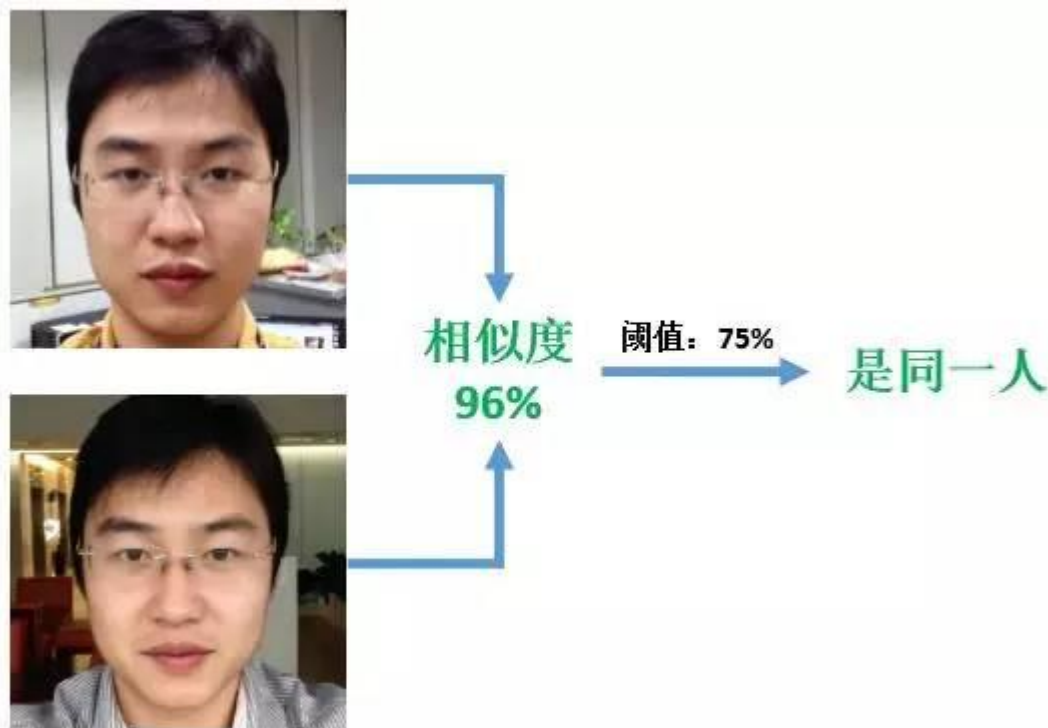
人脸识别概述

- ⌚ 人脸对比(Face Compare): 衡量两个人脸之间相似度的算法, 人脸对比是人脸特征提取出来的人脸特征基础上来进行的。



人脸识别概述

- ⌚ 人脸验证(Face Verification): 通过输入的两个人脸特征, 通过获取两个人脸特征之间的相似度, 然后基于预设的阈值来比较来判断这两个人脸特征是否属于同一个人。



人脸识别概述

- ⌚ 人脸识别(Face Recognition): 通过输入的人脸特征, 从数据库找出与输入特征相似度最高的用户, 然后进行用户信息判断。



MTCNN

⌚ MTCNN(Multi-task Cascaded Convolutional Networks, 多任务级联卷积神经网络), 将人脸区域检测和人脸关键点放到一起, 主体结构为级联结构, 主要由三个子网络P-Net、R-Net以及O-Net构成。

⌚ P-Net(Proposal Networks): 候选框快速生成;

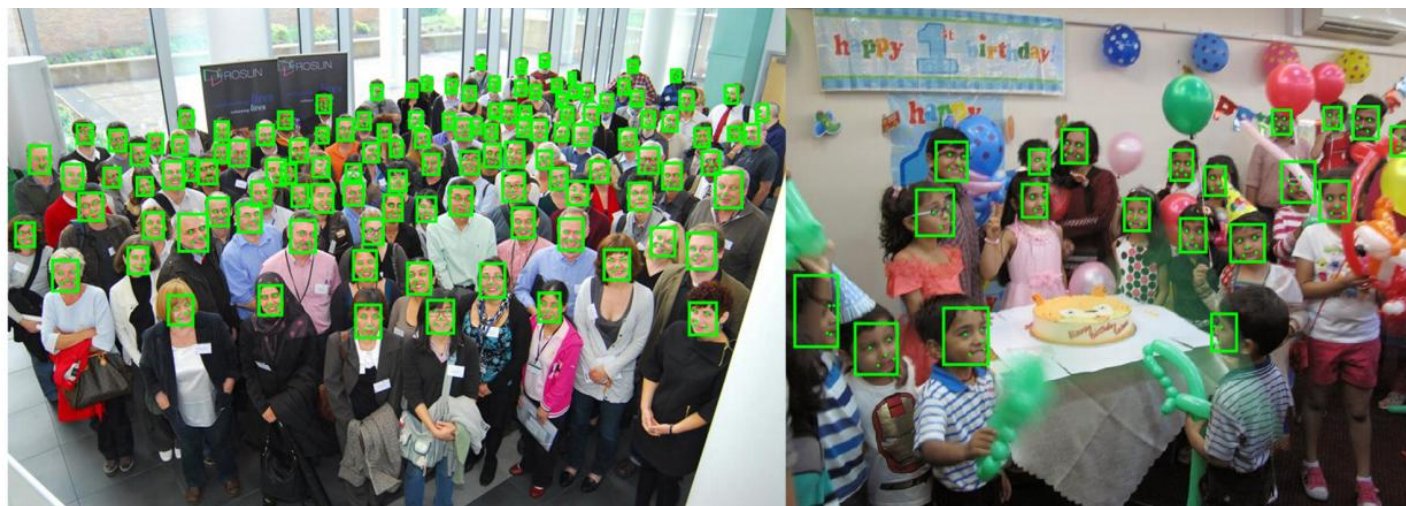
⌚ R-Net(Refine Networks): 高精度候选框过滤;

⌚ O-Net(Output Networks): 边框生成以及人脸关键点检测;

⌚ 图像金子塔、边框回归、非极大值抑制(NMS)等。

⌚ <https://arxiv.org/ftp/arxiv/papers/1604/1604.02878.pdf>

MTCNN



MTCNN

图像金字塔

R-Net、
Bounding Box、
NMS

P-Net、
Bounding Box、
NMS

O-Net、
Bounding Box、
NMS



Test image

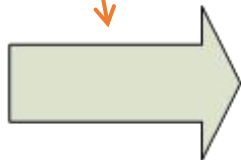
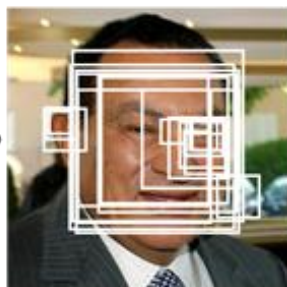
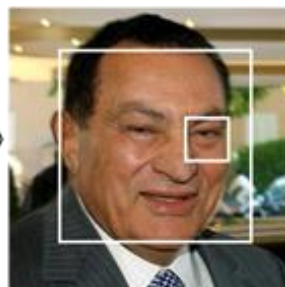
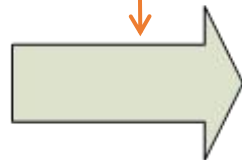


Image pyramid



Stage 1



Stage 2



Stage 3

MTCNN

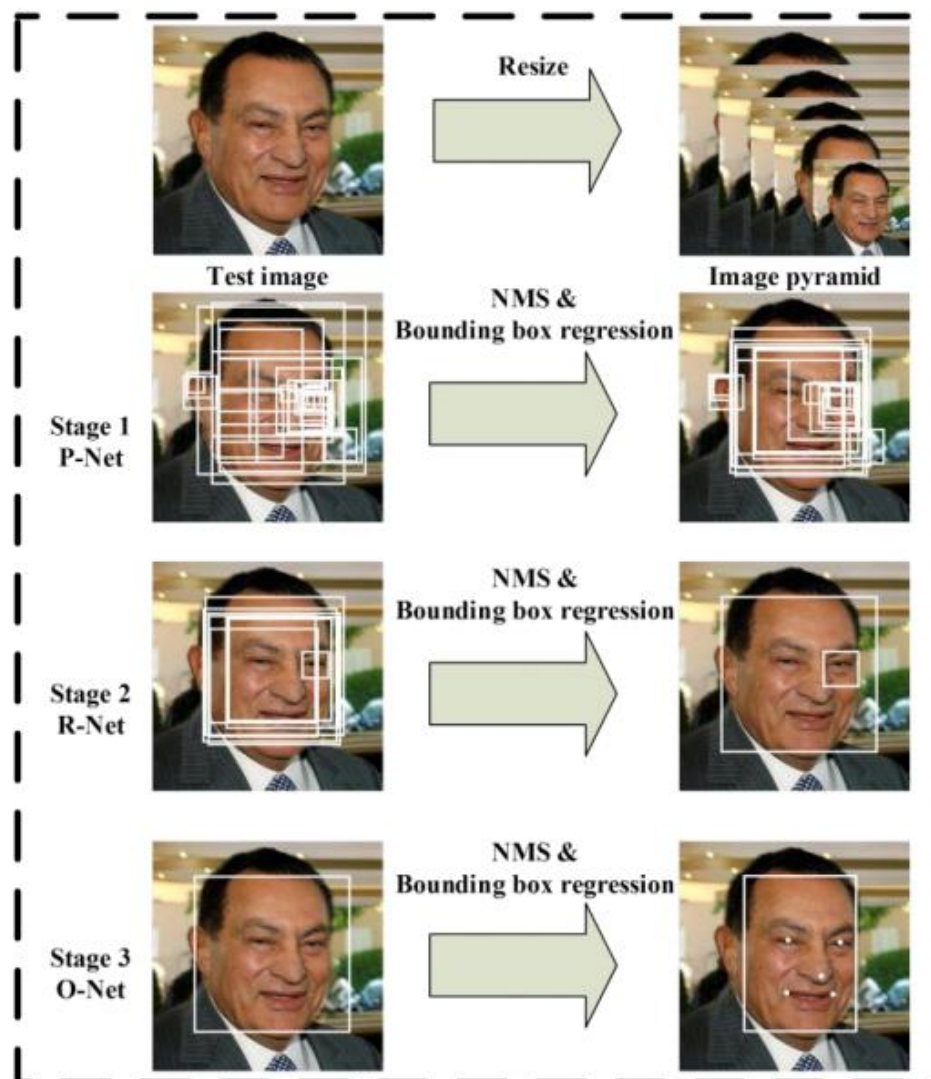
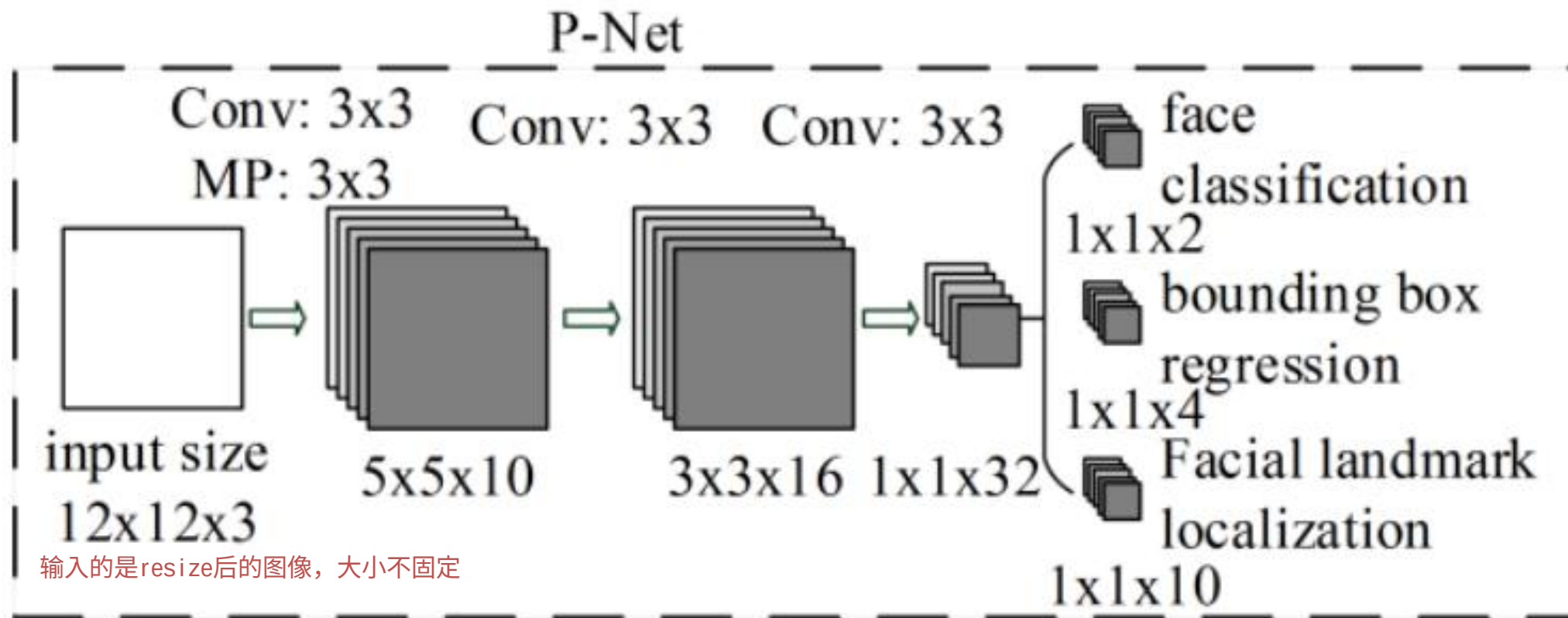


Fig. 1. Pipeline of our cascaded framework that includes three-stage multi-task deep convolutional networks. Firstly, candidate windows are produced through a fast Proposal Network (P-Net). After that, we refine these candidates in the next stage through a Refinement Network (R-Net). In the third stage, The Output Network (O-Net) produces final bounding box and facial landmarks position.

- ⌚ P-Net: Proposal Network, 基本结构就是一个全卷积网络, 对于上一步构建的图像金字塔, 通过FCN进行初步的特征提取以及边框标定, 并进行Bounding Box Regression回归调整边框位置以及NMS进行大部分窗口的过滤;
- ⌚ P-Net是一个人脸区域的区域建议网络, 属于一个浅层网络结构, 目的是**为了快速的产生人脸候选框**, 该结构最终会输出多张可能存在人脸的人脸区域, 并将这些区域输入到R-Net中进行下一步处理。

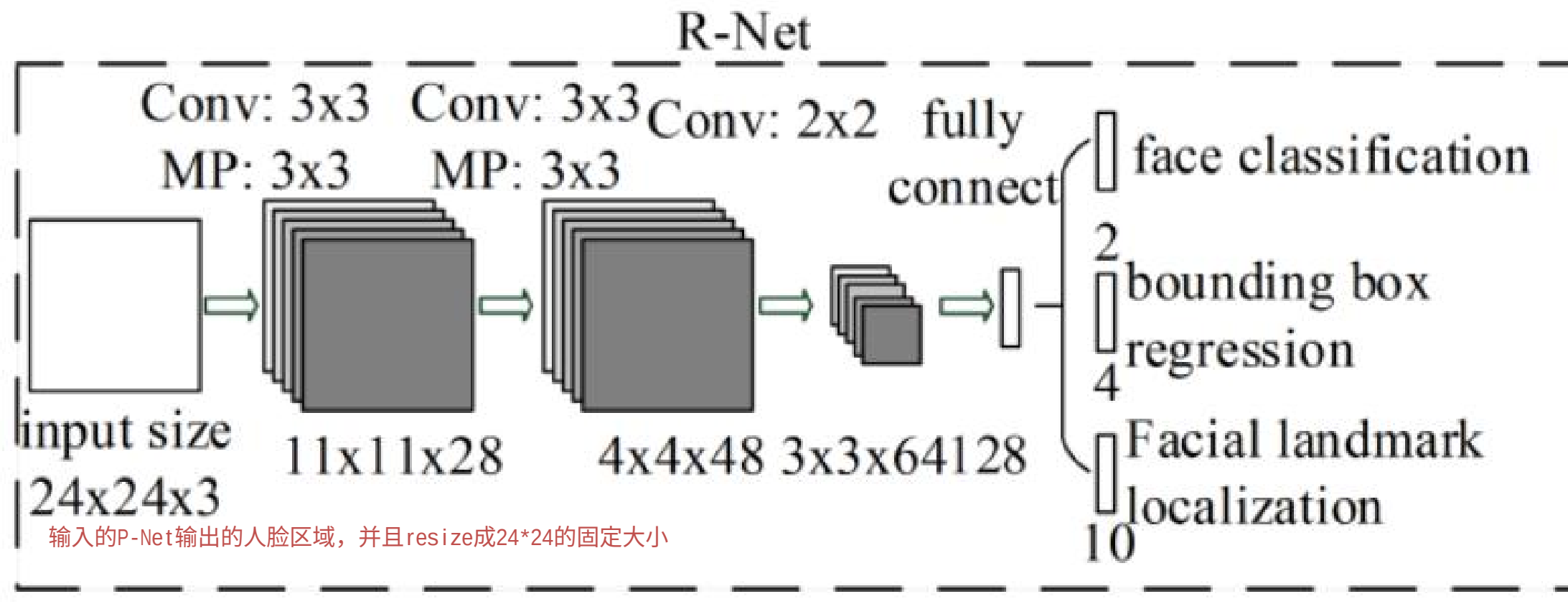
MTCNN



三层简单的卷积网络
三个分支：人脸分类、边框回归、关键点定位

- ⌚ R-Net: Refine Networks, 也属于简单的卷积网络结构, 相比于P-Net来讲, 增加一个全连接层, 因此对于输入数据的筛选会更加严格。在图片经过P-Net后, 会留下多个预测窗口, 将所有预测窗口输入R-Net中, 进一步过滤效果比较差的候选框, 最终对选定的候选框使用Bounding Box Regression和NMS进一步优化结果。
- ⌚ R-Net中输入的是具有一定可信度的人脸区域(也就是P-Net输出的人脸区域图像经过resize后的), 输出为可信度相对来讲比较高的人脸区域, 并且进行BBR和NMS优化。

MTCNN

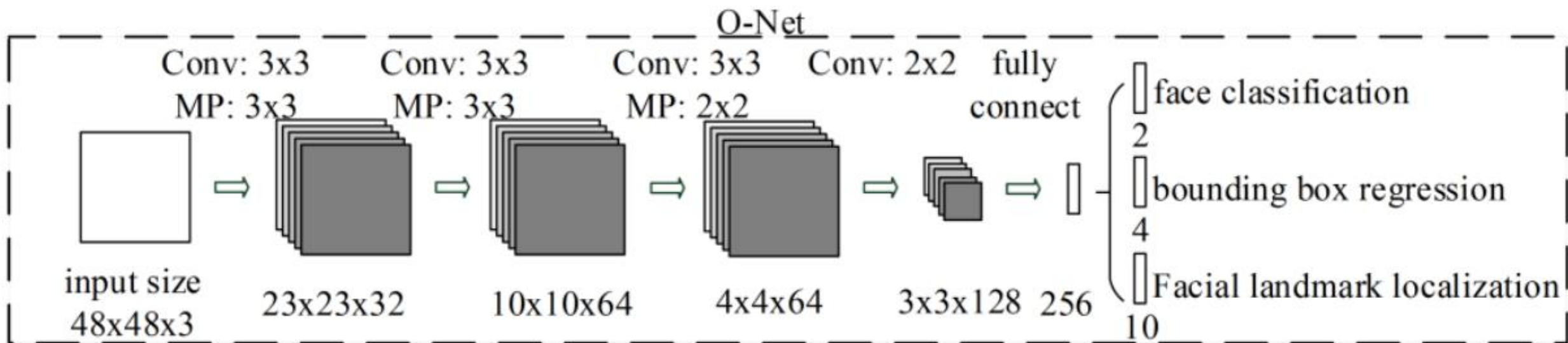


三层卷积+一层全连接的网络
三个分支：人脸分类、边框回归、关键点定位

MTCNN

- ⌚ O-Net: Output Networks, 基本结构是一个比较复杂的卷积神经网络, 相比于P-Net和R-Net来讲, 使用更加复杂的网络结构来保留更改的特征信息, 从而对于得到人脸的区域定位以及最终的关键点坐标信息。
- ⌚ O-Net和R-Net以及P-Net的结构类似, 主要区别是使用更加复杂的网络对模型性能做优化。

MTCNN

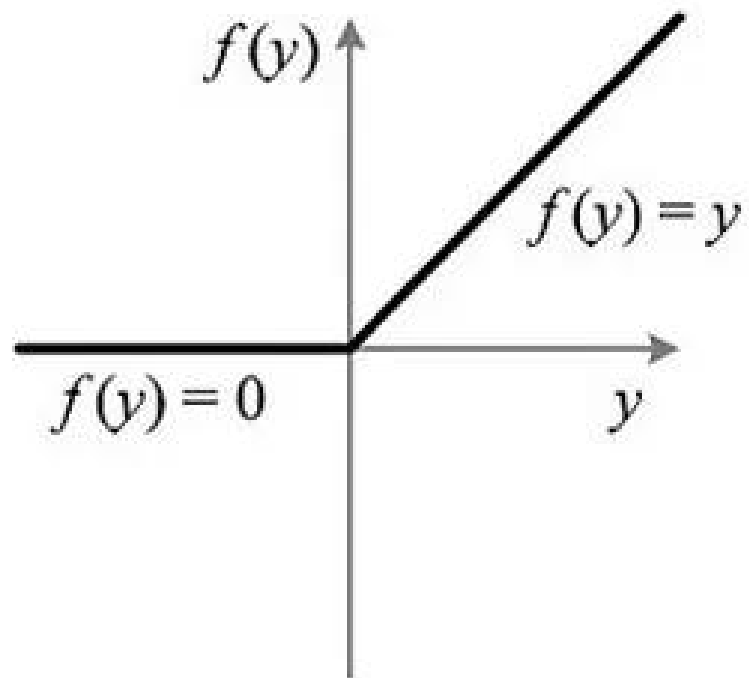


输入的R-Net输出的人脸区域，并且resize成48*48的固定大小

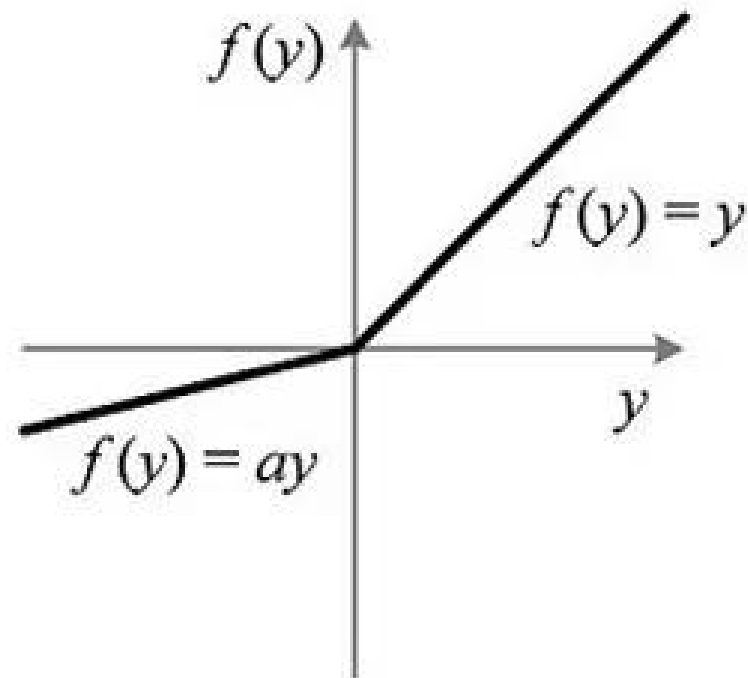
四层卷积+一层全连接的网络
三个分支：人脸分类、边框回归、关键点定位

- ⌚ 对于CNN网络结构，影响网络性能因素的主要原因有两个：
 - ⌚ 样本的多样性缺失会影响网络的鉴别能力；
 - ⌚ 相比于其他的分类检测网络，人脸检测属于一个二分类，每一层不需要太多的filter。
- ⌚ 结合上述原因，作者设计了每层的filter数量，将 $5*5$ 的卷积更换为两个 $3*3$ 的卷积核，这样相比于原始网络主要具有显著减少计算量、提升网络性能的效果，同时在网络中将所有的激活函数更改为PReLU。

MTCNN



$$ReLU(x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$



$$PReLU(x_i) = \begin{cases} x_i & \text{if } x_i > 0 \\ a_i x_i & \text{if } x_i \leq 0 \end{cases}$$

i 表示不同的通道

MTCNN

⌚ Face Classification: 二分类问题(是否属于人脸区域), 使用交叉熵损失函数。

$$L_i^{det} = -(y_i^{det} \log(p_i) + (1 - y_i^{det})(1 - \log(p_i)))$$

p_i 为网络预测当前区域属于人脸区域的概率值;
 y_i^{det} 为当前区域实际上是否包含人脸,
0表示不包含, 1表示包含

MTCNN

- ⌚ Bounding Box Regression: 直接预测边框坐标，这里采用直接预测边框坐标的主要目的是为了快速的获取边框，属于一个回归问题，故采用MSE距离损失函数。

$$L_i^{box} = \|\hat{y}_i^{box} - y_i^{box}\|_2^2$$

预测边框: (x, y, height, width)

实际边框: (x, y, height, width)

MTCNN

- ⌚ Facial landmark localization: 和BBR一样, Facial landmark localization的也是一个回归模型, 其主要预测值为5点的坐标, 分别为: left eye、right eye、nose、left mouth corner、right mouth corner; 总共由5个坐标值组成的10维向量。

$$L_i^{landmark} = \|\hat{y}_i^{landmark} - y_i^{landmark}\|_2^2$$

预测坐标

实际坐标

MTCNN

- ⌚ MTCNN中模型的损失函数是由分类、回归以及定位三部分损失线性组合而来的。

$$\min \sum_{i=1}^N \sum_{j \in \{det, box, landmark\}} \alpha_j \beta_i^j L_i^j \quad (4)$$

alpha阈值加大的主要原因是为了让定位越正确。为了获取更加精确的脸部标记点位置点信息。

where N is the number of training samples. α_j denotes on the task importance. We use $(\alpha_{det} = 1, \alpha_{box} = 0.5, \alpha_{landmark} = 0.5)$ in P-Net and R-Net, while $(\alpha_{det} = 1, \alpha_{box} = 0.5, \alpha_{landmark} = 1)$ in O-Net for more accurate facial landmarks localization. $\beta_i^j \in \{0,1\}$ is the sample type indicator. In this case, it is natural to employ stochastic gradient descent to train the CNNs.

⌚ Online Hard Sample Mining(在线难样本挖掘算法):

- ⌚ 在模型训练的过程中，对于每个批次的样本而言，不是使用所有样本的损失进行模型更新的，而是使用70%的损失函数值最高的样本来反向传播的，原因是：
好的样本对于网络的提升效果有限，只有那些难样本更加有效训练，进行反向传播之后才可以更高的提升网络效果。

MTCNN

⌚ Negatives:

⌚ $\text{IoU} < 0.3$

⌚ Positives:

⌚ $\text{IoU} > 0.65$

⌚ Part Faces:

⌚ $\text{IoU}: [0.4, 0.65]$

⌚ Landmark Faces:

⌚ faces labeled 5 landmark's positions

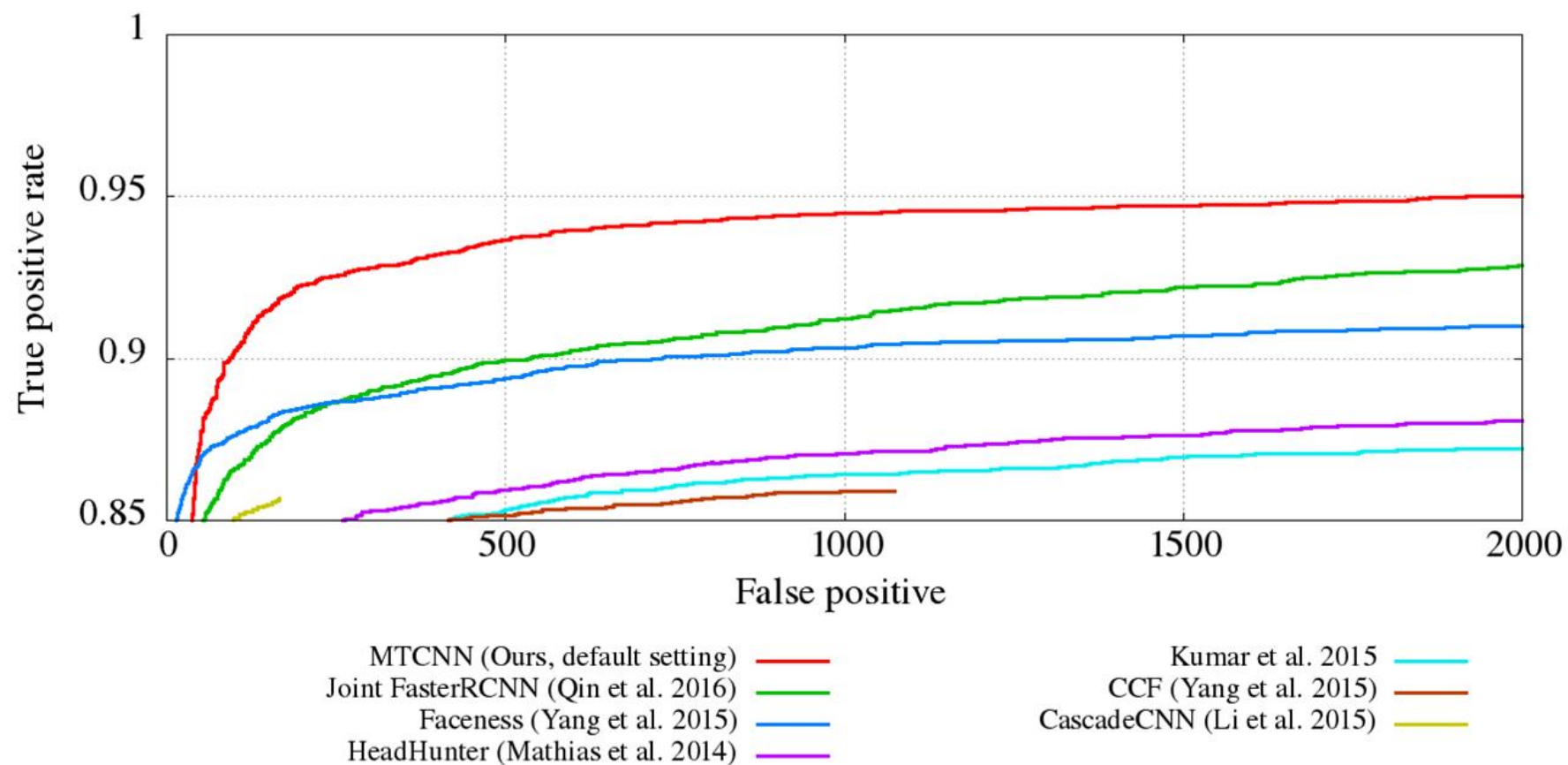
分类模型

回归模型

定位模型



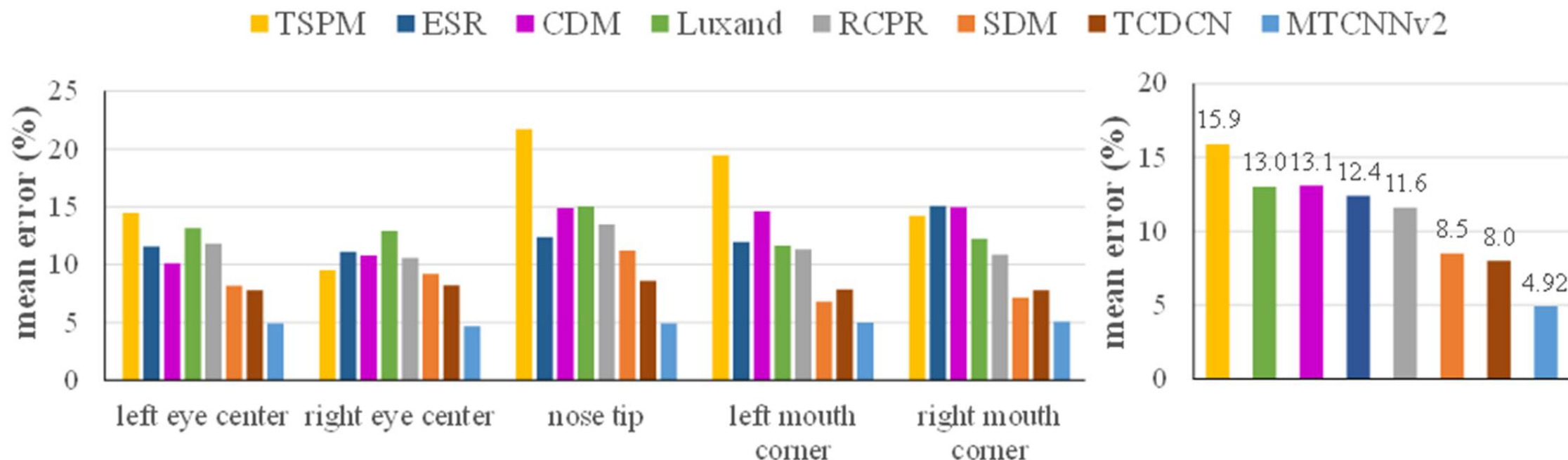
MTCNN



(c) Result on FDBB for face detection

MTCNN

1. 整个图像resize成不同的尺度，得到多个图像
2. 将每个resize后的图像，输入到P-Net中，得到m个预测边框(coef + box),边框位置需要还原到原始图像上。
3. 针对所有的预测边框，进行coef阈值过滤 + NMS非极大值过滤,得到剩余的预测边框；
4. 将每个预测边框在真实图像上的对应区域截取出来，并resize成24*24的大小，输入到R-Net中，得到当前输入边框对应的coef + box；
5. 将所有转换后的边框进行新一轮的coef阈值过滤 + NMS非极大值过滤，得到剩余的预测边框；
6. 将上一步输出的每个预测边框在真实图像上的对应区域提取出来，并resize成48*48的大小，输入到O-Net中，得到当前输入边框对应的coef + box + keypoints；
7. 将上一步的所有边框，进行新一轮的coef阈值过滤 + NMS非极大值处理，得到最终预测边框；



(d) Result on AFLW for face alignment

特征向量提取

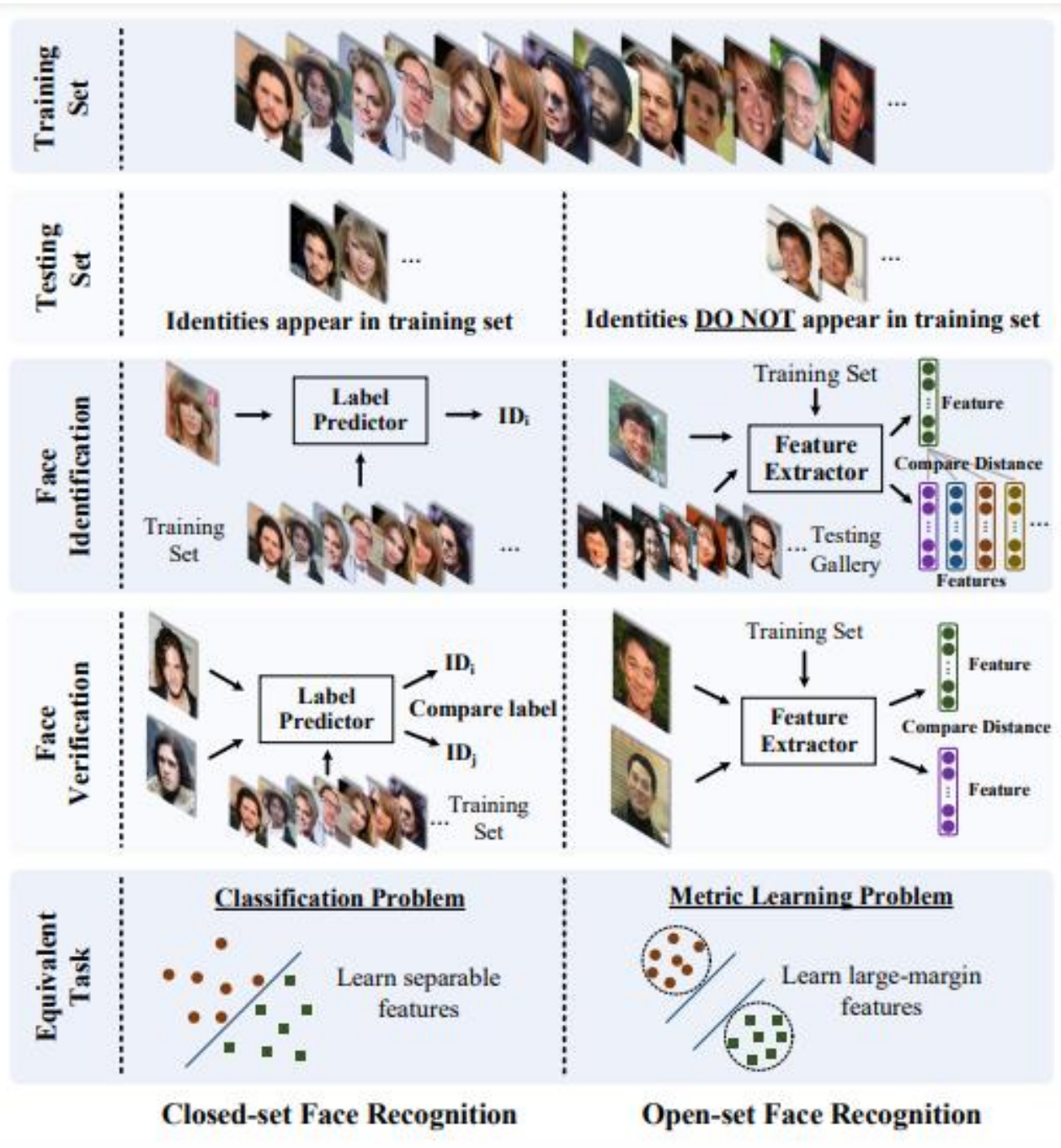
底层是分类模型，相当于只获取分类模型最后一层特征向量，不需要分类结果。

----->

训练时候输入x就是原始图像image，输入y就是类别标签id

----->

y实际上就是人的ID

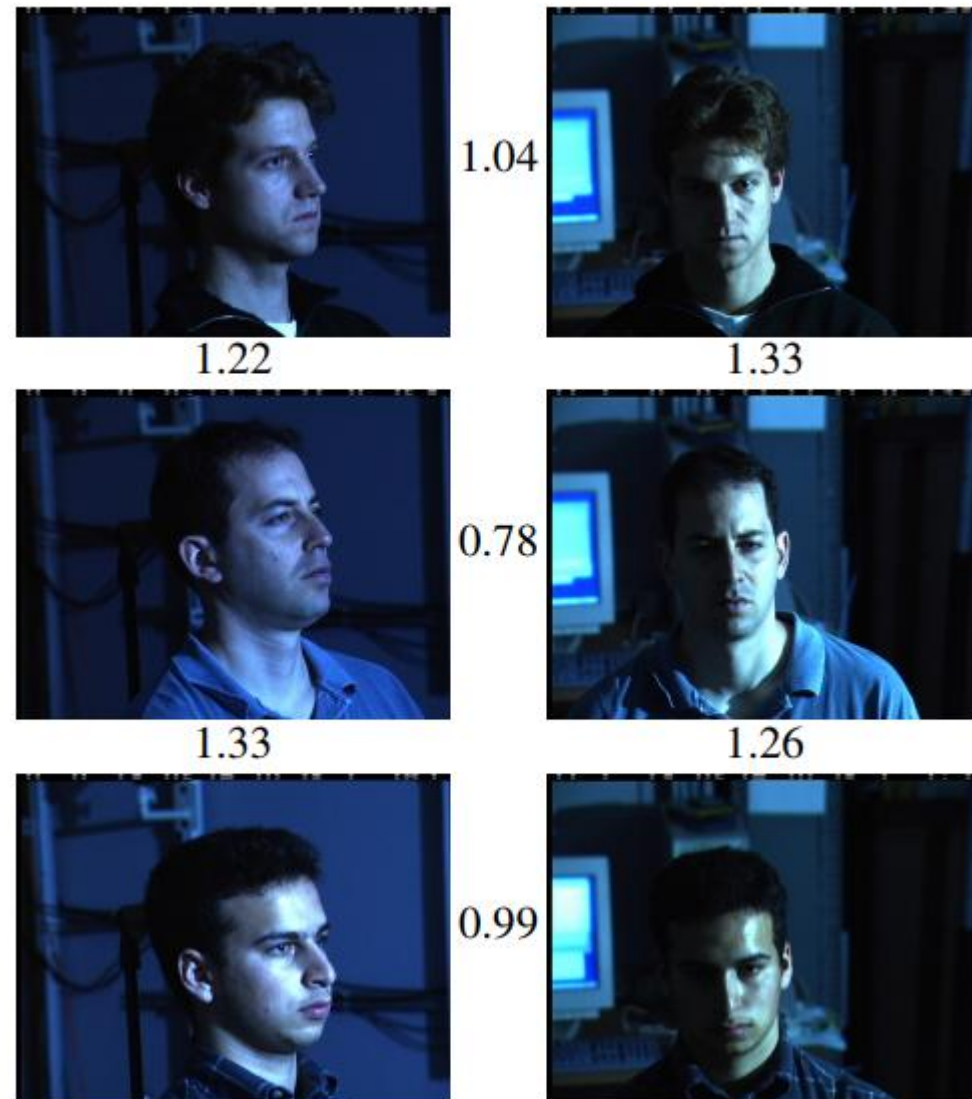


训练目标:
类别数据分割开

训练目标:
类别数据分割开 + 同一类别的向量要尽可能的集中
--> 同一个类别的向量要尽可能的集中, 不同类别的向量要尽可能的远离

Face Net

- ⌚ FaceNet(A Unified Embedding for Face Recognition and Clustering),
<https://arxiv.org/pdf/1503.03832.pdf>
- ⌚ 通过 CNN 将人脸映射到欧式空间的特征向量上，计算不同图片人脸特征的距离，通过相同个体的人脸的距离，总是小于不同个体的人脸这一先验知识训练网络。



Face Net

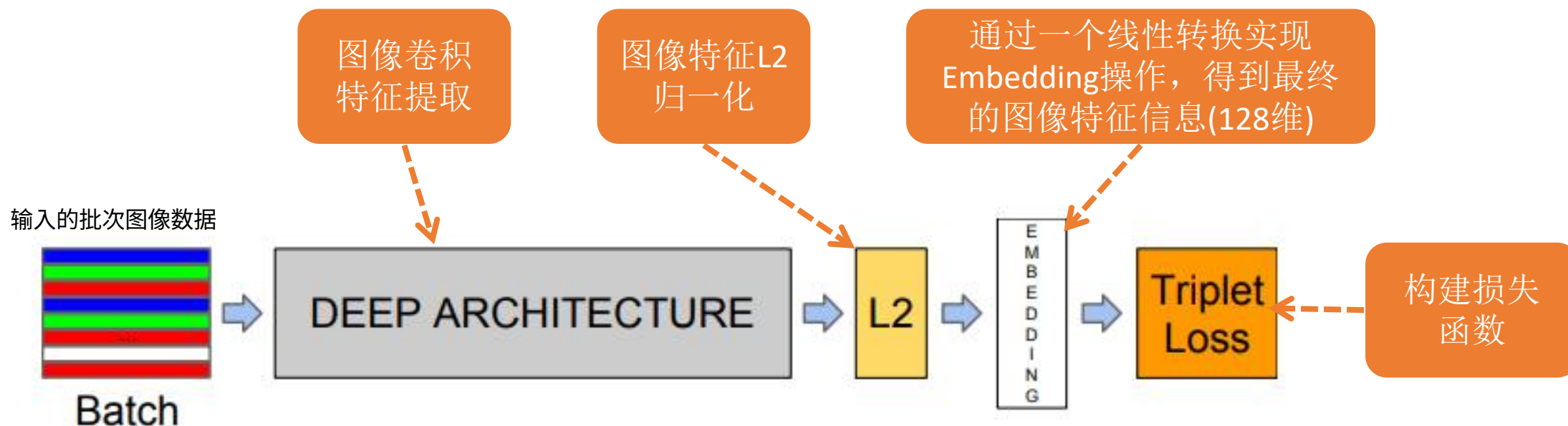


Figure 2. **Model structure.** Our network consists of a batch input layer and a deep CNN followed by L_2 normalization, which results in the face embedding. This is followed by the triplet loss during training.

Face Net

⌚ 存在特征向量的维度选择问题，维度约小计算越快，但是太小的话很难区分不同图片；维度越大越容易区分不同图片，但是太大训练模型不易收敛，且测试时计算慢，占用空间大。作者实验证明 128 维的特征能够较好的平衡这个问题。

#dims	VAL
64	86.8% $\pm$ 1.7
128	87.9% $\pm$ 1.9
256	87.7% $\pm$ 1.9
512	85.6% $\pm$ 2.0

Table 5. **Embedding Dimensionality.** This Table compares the effect of the embedding dimensionality of our model NN1 on our hold-out set from section 4.1. In addition to the VAL at 10E-3 we also show the standard error of the mean computed across five splits.

Face Net

⌚ 三元损失函数(Triplet Loss)

- ⌚ 二元损失函数的目标是把相同个体的人脸特征映射到空间中的相同点。
- ⌚ 三元损失函数目标是映射到相同的区域，使得**类内距离小于类间距离(聚类)**。

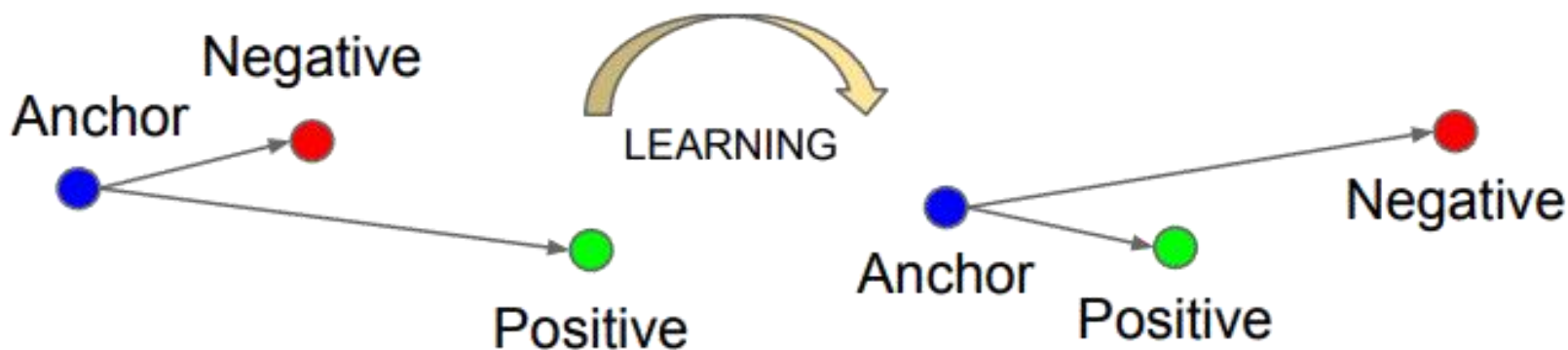
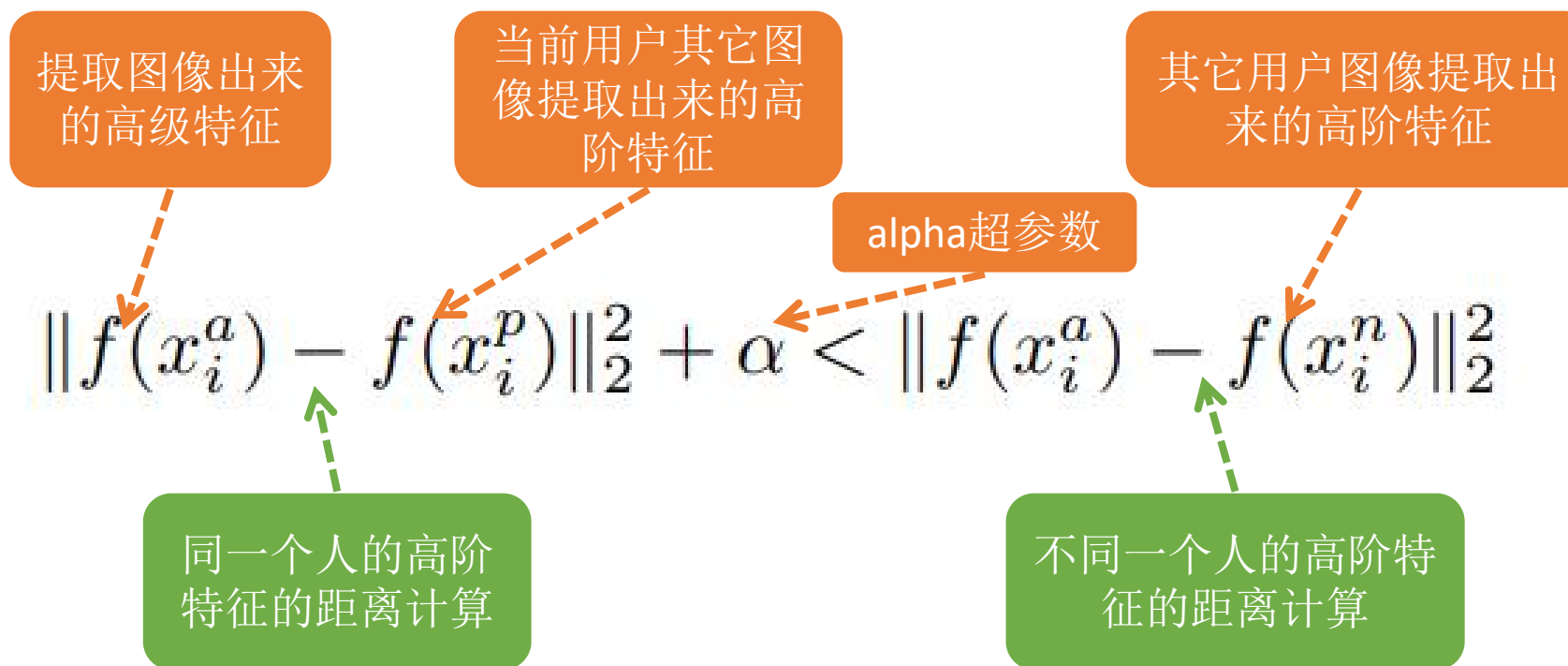
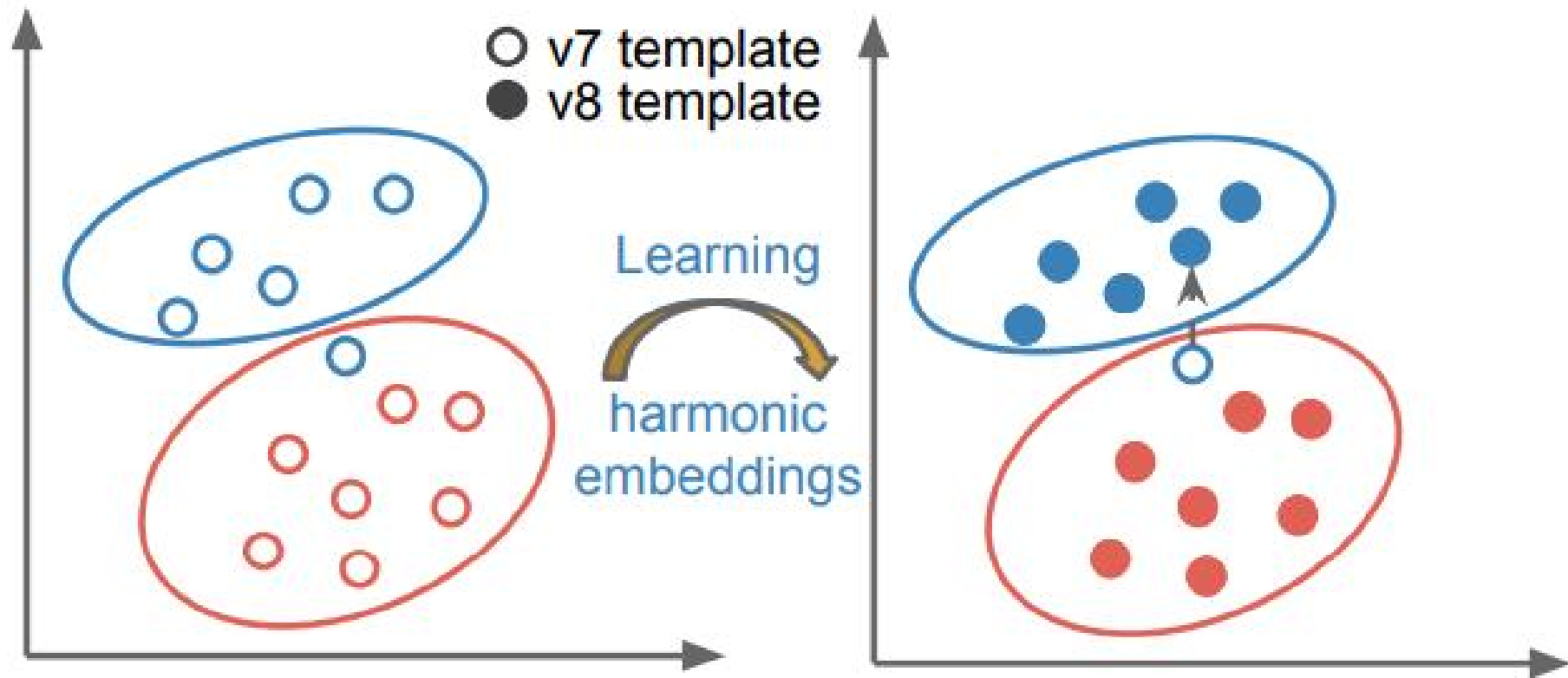


Figure 3. The **Triplet Loss** minimizes the distance between an *an-chor* and a *positive*, both of which have the same identity, and maximizes the distance between the *anchor* and a *negative* of a different identity.

Face Net



Face Net



Face Net

⌚ 损失函数:

- ⌚ 实际上不是使用所有样本来计算损失，在同一个类别中，选择距离最远的样本(hard positive)，在不同类别中，选择距离最近的样本(hard negative)的距离来构造损失函数。

$$\sum_i^N \left[\|f(x_i^a) - f(x_i^p)\|_2^2 - \|f(x_i^a) - f(x_i^n)\|_2^2 + \alpha \right]$$

选择同一个人脸
的不同图像上距
离最远的

选择不同一个人
脸图像上距离最
近的

Face Net

⌚ 网络结构：基于VGGNet

layer	size-in	size-out	kernel	param	FLPS
conv1	$220 \times 220 \times 3$	$110 \times 110 \times 64$	$7 \times 7 \times 3, 2$	9K	115M
pool1	$110 \times 110 \times 64$	$55 \times 55 \times 64$	$3 \times 3 \times 64, 2$	0	
rnorm1	$55 \times 55 \times 64$	$55 \times 55 \times 64$		0	
conv2a	$55 \times 55 \times 64$	$55 \times 55 \times 64$	$1 \times 1 \times 64, 1$	4K	13M
conv2	$55 \times 55 \times 64$	$55 \times 55 \times 192$	$3 \times 3 \times 64, 1$	111K	335M
rnorm2	$55 \times 55 \times 192$	$55 \times 55 \times 192$		0	
pool2	$55 \times 55 \times 192$	$28 \times 28 \times 192$	$3 \times 3 \times 192, 2$	0	
conv3a	$28 \times 28 \times 192$	$28 \times 28 \times 192$	$1 \times 1 \times 192, 1$	37K	29M
conv3	$28 \times 28 \times 192$	$28 \times 28 \times 384$	$3 \times 3 \times 192, 1$	664K	521M
pool3	$28 \times 28 \times 384$	$14 \times 14 \times 384$	$3 \times 3 \times 384, 2$	0	
conv4a	$14 \times 14 \times 384$	$14 \times 14 \times 384$	$1 \times 1 \times 384, 1$	148K	29M
conv4	$14 \times 14 \times 384$	$14 \times 14 \times 256$	$3 \times 3 \times 384, 1$	885K	173M
conv5a	$14 \times 14 \times 256$	$14 \times 14 \times 256$	$1 \times 1 \times 256, 1$	66K	13M
conv5	$14 \times 14 \times 256$	$14 \times 14 \times 256$	$3 \times 3 \times 256, 1$	590K	116M
conv6a	$14 \times 14 \times 256$	$14 \times 14 \times 256$	$1 \times 1 \times 256, 1$	66K	13M
conv6	$14 \times 14 \times 256$	$14 \times 14 \times 256$	$3 \times 3 \times 256, 1$	590K	116M
pool4	$14 \times 14 \times 256$	$7 \times 7 \times 256$	$3 \times 3 \times 256, 2$	0	
concat	$7 \times 7 \times 256$	$7 \times 7 \times 256$		0	
fc1	$7 \times 7 \times 256$	$1 \times 32 \times 128$	maxout p=2	103M	103M
fc2	$1 \times 32 \times 128$	$1 \times 32 \times 128$	maxout p=2	34M	34M
fc7128	$1 \times 32 \times 128$	$1 \times 1 \times 128$		524K	0.5M
L2	$1 \times 1 \times 128$	$1 \times 1 \times 128$		0	
total				140M	1.6B

Table 1. **NN1**. This table show the structure of our Zeiler&Fergus [22] based model with 1×1 convolutions inspired by [9]. The input and output sizes are described in $rows \times cols \times \#filters$. The kernel is specified as $rows \times cols, stride$ and the maxout [6] pooling size as $p = 2$.

Face Net

🕒 网络结构：基于GoogleNet

type	output size	depth	#1×1	#3×3 reduce	#3×3	#5×5 reduce	#5×5	pool proj (p)	params	FLOPS
conv1 (7×7×3, 2)	112×112×64	1							9K	119M
max pool + norm	56×56×64	0						m 3×3, 2		
inception (2)	56×56×192	2		64	192				115K	360M
norm + max pool	28×28×192	0						m 3×3, 2		
inception (3a)	28×28×256	2	64	96	128	16	32	m, 32p	164K	128M
inception (3b)	28×28×320	2	64	96	128	32	64	L_2 , 64p	228K	179M
inception (3c)	14×14×640	2	0	128	256, 2	32	64, 2	m 3×3, 2	398K	108M
inception (4a)	14×14×640	2	256	96	192	32	64	L_2 , 128p	545K	107M
inception (4b)	14×14×640	2	224	112	224	32	64	L_2 , 128p	595K	117M
inception (4c)	14×14×640	2	192	128	256	32	64	L_2 , 128p	654K	128M
inception (4d)	14×14×640	2	160	144	288	32	64	L_2 , 128p	722K	142M
inception (4e)	7×7×1024	2	0	160	256, 2	64	128, 2	m 3×3, 2	717K	56M
inception (5a)	7×7×1024	2	384	192	384	48	128	L_2 , 128p	1.6M	78M
inception (5b)	7×7×1024	2	384	192	384	48	128	m, 128p	1.6M	78M
avg pool	1×1×1024	0								
fully conn	1×1×128	1							131K	0.1M
L2 normalization	1×1×128	0								
total									7.5M	1.6B

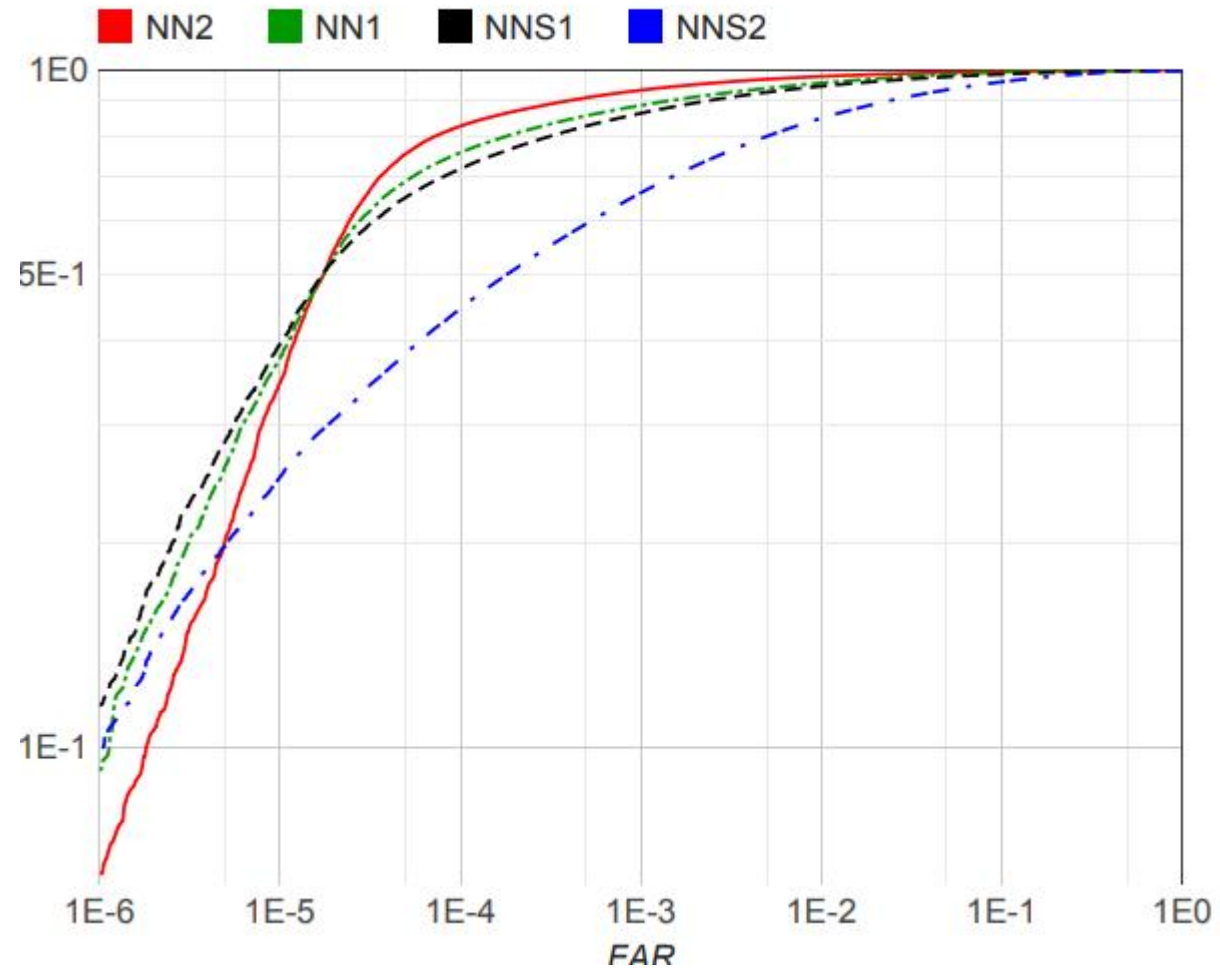
Table 2. **NN2**. Details of the NN2 Inception incarnation. This model is almost identical to the one described in [16]. The two major differences are the use of L_2 pooling instead of max pooling (m), where specified. *I.e.* instead of taking the spatial max the L_2 norm is computed. The pooling is always 3×3 (aside from the final average pooling) and in parallel to the convolutional modules inside each Inception module. If there is a dimensionality reduction after the pooling it is denoted with p. 1×1, 3×3, and 5×5 pooling are then concatenated to get the final output.

Face Net

architecture	VAL
NN1 (Zeiler&Fergus 220×220)	$87.9\% \pm 1.9$
NN2 (Inception 224×224)	$89.4\% \pm 1.6$
NN3 (Inception 160×160)	$88.3\% \pm 1.7$
NN4 (Inception 96×96)	$82.0\% \pm 2.3$
NNS1 (mini Inception 165×165)	$82.4\% \pm 2.4$
NNS2 (tiny Inception 140×116)	$51.9\% \pm 2.9$

Table 3. **Network Architectures.** This table compares the performance of our model architectures on the hold out test set (see section 4.1). Reported is the mean validation rate VAL at $10E-3$ false accept rate. Also shown is the standard error of the mean across the five test splits.

Face Net



扩展: SphereFace

🕒 SphereFace构建了一个在softmax loss的基础上引入角度信息的损失，通过**乘性角度边距惩罚 (multiplicative angular margin penalty)**最终增加同类数据的内聚性；

$$L_{\text{modified}} = \frac{1}{N} \sum_i -\log \left(\frac{e^{\|\mathbf{x}_i\| \cos(\theta_{y_i,i})}}{\sum_j e^{\|\mathbf{x}_i\| \cos(\theta_{j,i})}} \right)$$

🕒 <https://arxiv.org/pdf/1704.08063.pdf>
https://zhuanlan.zhihu.com/p/95447796?utm_id=0

NOTE: 直接对w进行L2-Norm处理, 从而让w的二范式等于1, 将所有参数b直接置为0.

$$L_i = -\log \left(\frac{e^{\mathbf{W}_{y_i}^T \mathbf{x}_i + b_{y_i}}}{\sum_j e^{\mathbf{W}_j^T \mathbf{x}_i + b_j}} \right)$$
$$= -\log \left(\frac{e^{\|\mathbf{W}_{y_i}\| \|\mathbf{x}_i\| \cos(\theta_{y_i,i}) + b_{y_i}}}{\sum_j e^{\|\mathbf{W}_j\| \|\mathbf{x}_i\| \cos(\theta_{j,i}) + b_j}} \right)$$

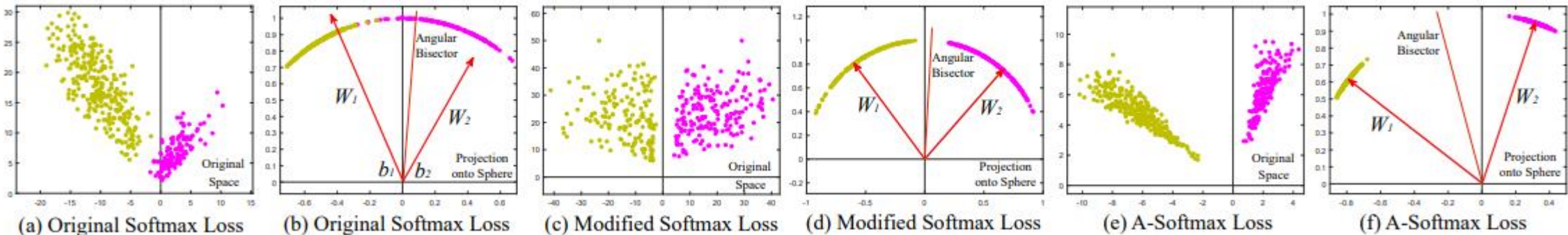
Loss Function	Decision Boundary
Softmax Loss	$(\mathbf{W}_1 - \mathbf{W}_2)\mathbf{x} + b_1 - b_2 = 0$
Modified Softmax Loss	$\ \mathbf{x}\ (\cos \theta_1 - \cos \theta_2) = 0$
A-Softmax Loss	$\ \mathbf{x}\ (\cos m\theta_1 - \cos \theta_2) = 0$ for class 1 $\ \mathbf{x}\ (\cos \theta_1 - \cos m\theta_2) = 0$ for class 2

$$L_{\text{ang}} = \frac{1}{N} \sum_i -\log \left(\frac{e^{\|\mathbf{x}_i\| \cos(m\theta_{y_i,i})}}{e^{\|\mathbf{x}_i\| \cos(m\theta_{y_i,i})} + \sum_{j \neq y_i} e^{\|\mathbf{x}_i\| \cos(\theta_{j,i})}} \right) \quad (6)$$

NOTE: 在求解角度 θ 的关系的时候, 加入惩罚性m, 从而让属于同一个类别的样本之间的角度更小。要求 $m > 1$, 一般 $m=4$;

NOTE: 全连接转换成余弦角度的计算, 并且损失函数进行更改 --> 本身的Softmax Loss求解的就是角度的大小关系。

Table 1: Comparison of decision boundaries in binary case. Note that, θ_i is the angle between $\mathbf{W}_i$ and $\mathbf{x}$.



扩展: ArcFace

$$L_1 = -\log \frac{e^{W_{y_i}^T x_i + b_{y_i}}}{\sum_{j=1}^N e^{W_j^T x_i + b_j}},$$

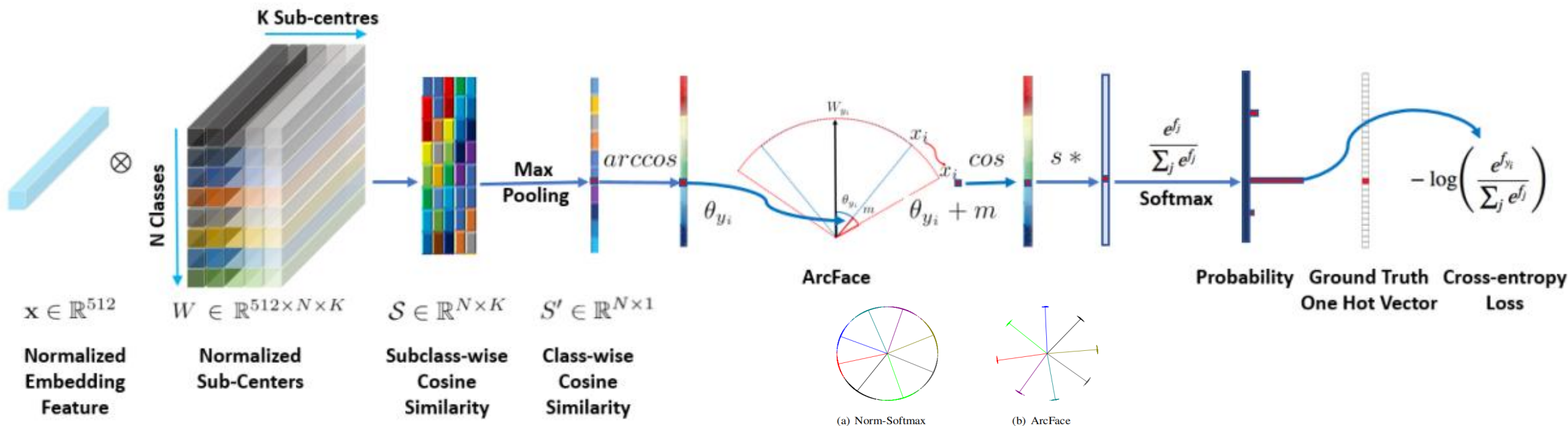
$$L_2 = -\log \frac{e^{s \cos \theta_{y_i}}}{e^{s \cos \theta_{y_i}} + \sum_{j=1, j \neq y_i}^N e^{s \cos \theta_j}}.$$

⌚ ArcFace构建了一个在softmax loss的基础上引入角度信息的损失，通过**加性角度边距惩罚 (additive angular margin penalty)** 最终增加同类数据的内聚性;

$$L_3 = -\log \frac{e^{s \cos(\theta_{y_i} + m)}}{e^{s \cos(\theta_{y_i} + m)} + \sum_{j=1, j \neq y_i}^N e^{s \cos \theta_j}}.$$

⌚ https://blog.csdn.net/qq_39478403/article/details/116788113

⌚ <https://arxiv.org/pdf/1801.07698.pdf>



扩展: CosFace

⌚ CosFace构建了一个在softmax loss的基础上引入角度信息的损失，通过**加性余弦边距惩罚 (additive cosine margin penalty)**最终增加同类数据的内聚性;

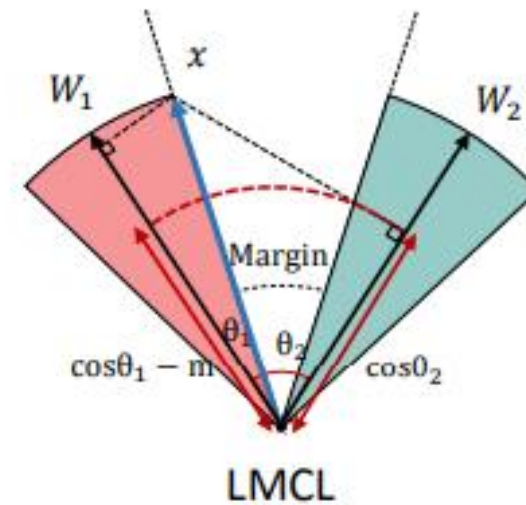
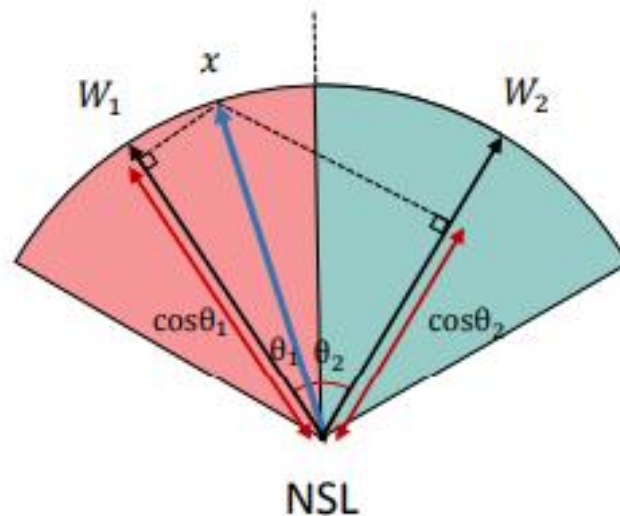
⌚ <https://zhuanlan.zhihu.com/p/45153595>

⌚ <https://arxiv.org/pdf/1801.09414.pdf>

$$L_{lmc} = \frac{1}{N} \sum_i -\log \frac{e^{s(\cos(\theta_{y_i,i})-m)}}{e^{s(\cos(\theta_{y_i,i})-m)} + \sum_{j \neq y_i} e^{s \cos(\theta_{j,i})}}, \quad (4)$$

subject to

$$\begin{aligned} W &= \frac{W^*}{\|W^*\|}, \\ x &= \frac{x^*}{\|x^*\|}, \\ \cos(\theta_{j,i}) &= W_j^T x_i, \end{aligned} \quad (5)$$

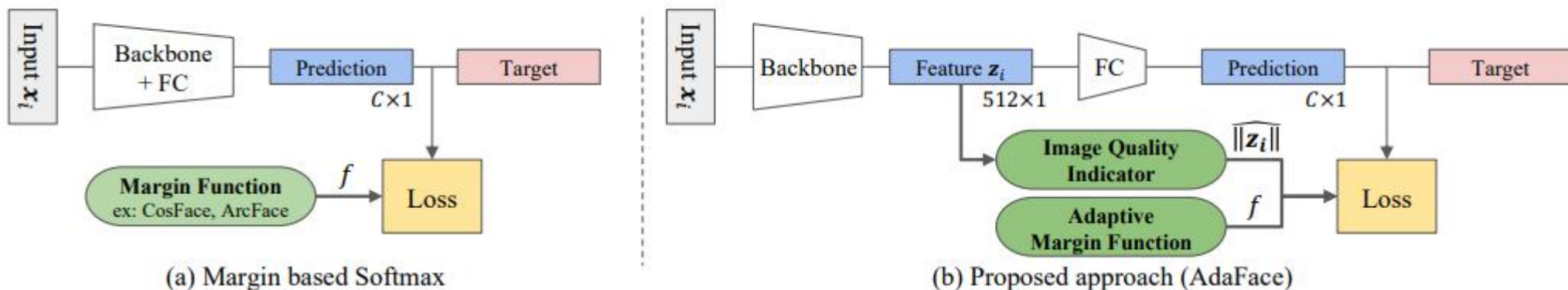


扩展：AdaFace

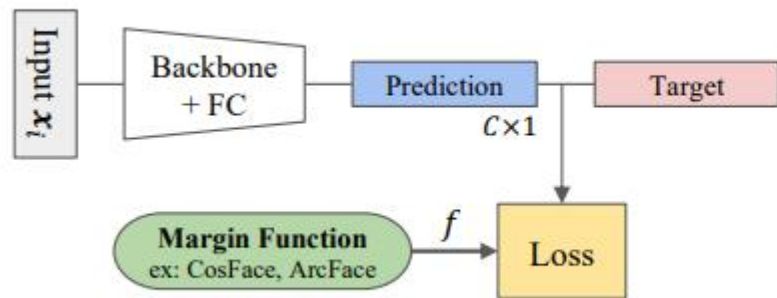
⌚ 虽然在CosFace、ArcFace等算法中通过对角度进行优化，让类间更加紧凑，但是在这些算法中边距惩罚值都是固定值，在一些低质量的数据集上效果可能会不太好，所以AdaFace提出了一种自适应的惩罚值计算方式。



https://openaccess.thecvf.com/content/CVPR2022/papers/Kim_AdaFace_Quality_Adaptive_Margin_for_Face_Recognition_CVPR_2022_paper.pdf



扩展: AdaFace



(a) Margin based Softmax

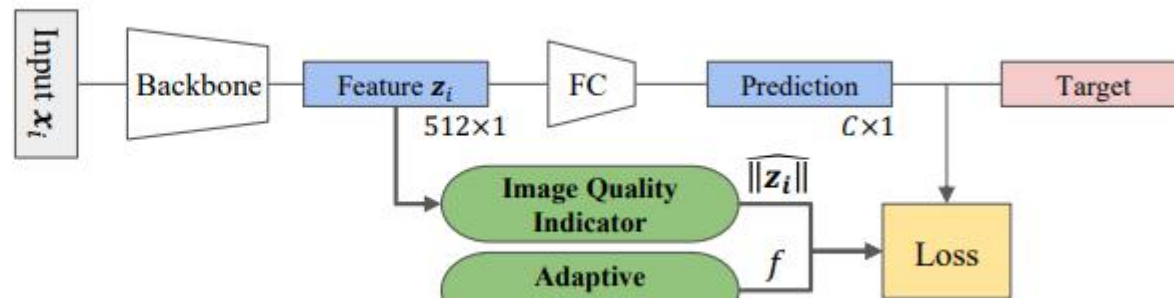
$$\mathcal{L} = -\log \frac{\exp(f(\theta_{y_i}, m))}{\exp(f(\theta_{y_i}, m)) + \sum_{j \neq y_i}^n \exp(s \cos \theta_j)}, \quad (1)$$

where θ_j is the angle between the feature vector and the j^{th} classifier weight vector, y_i is the index of the ground truth (GT) label, and m is the margin, which is a scalar hyper-parameter. f is a margin function, where

$$f(\theta_j, m)_{\text{SphereFace}} = \begin{cases} s \cos(m\theta_j) & j = y_i \\ s \cos \theta_j & j \neq y_i \end{cases}, \quad (2)$$

$$f(\theta_j, m)_{\text{CosFace}} = \begin{cases} s(\cos \theta_j - m) & j = y_i \\ s \cos \theta_j & j \neq y_i \end{cases}, \quad (3)$$

$$f(\theta_j, m)_{\text{ArcFace}} = \begin{cases} s \cos(\theta_j + m) & j = y_i \\ s \cos \theta_j & j \neq y_i \end{cases}. \quad (4)$$



(b) Proposed approach (AdaFace)

$$f(\theta_j, m)_{\text{AdaFace}} = \begin{cases} s \cos(\theta_j + g_{\text{angle}}) - g_{\text{add}} & j = y_i \\ s \cos \theta_j & j \neq y_i \end{cases}, \quad (18)$$

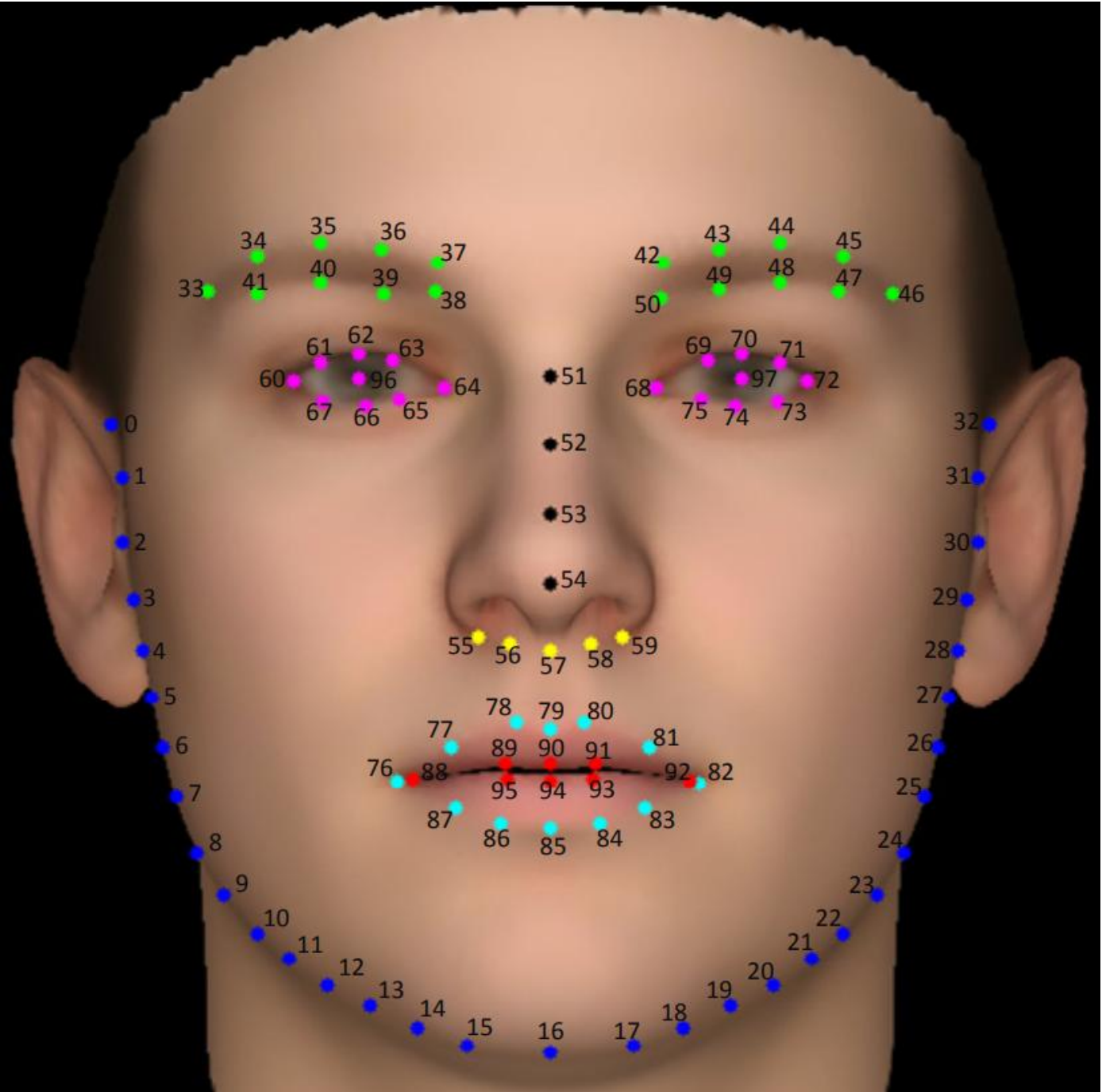
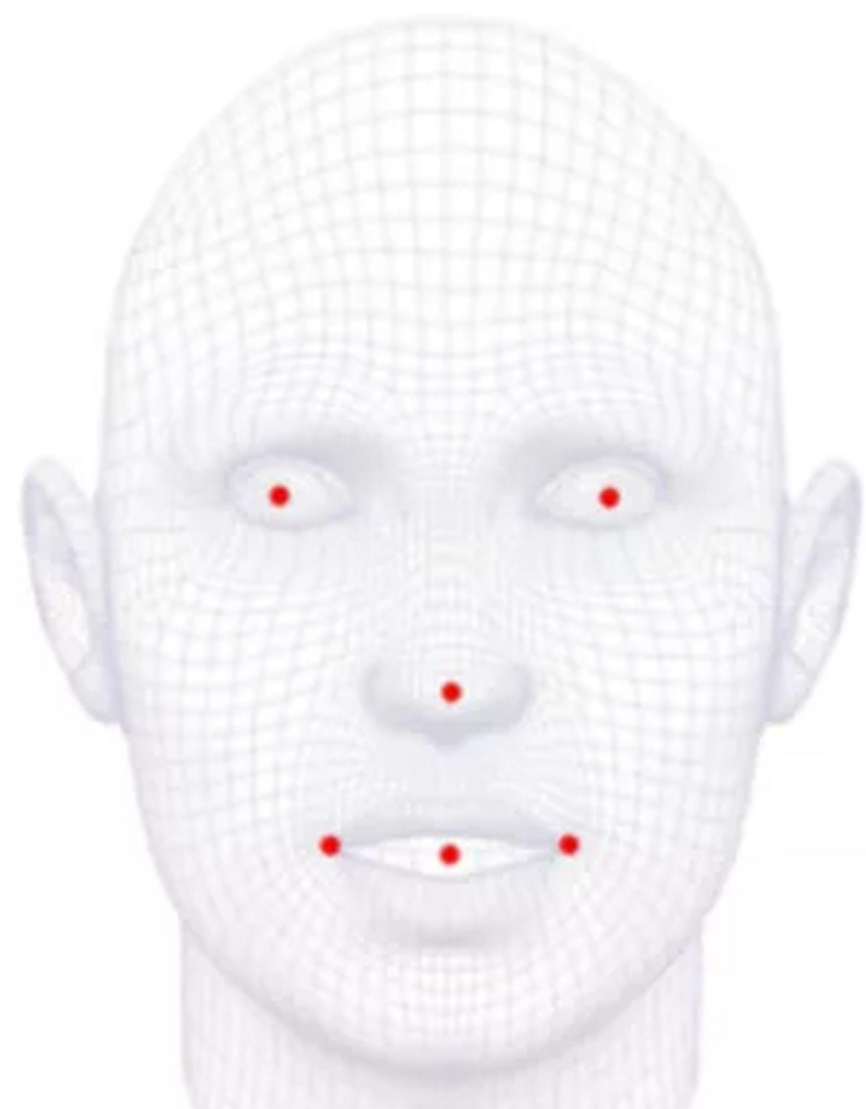
$$g_{\text{angle}} = -m \cdot \|\widehat{\mathbf{z}_i}\|, \quad g_{\text{add}} = m \cdot \|\widehat{\mathbf{z}_i}\| + m. \quad (19)$$

Note that when $\|\widehat{\mathbf{z}_i}\| = -1$, the proposed function becomes ArcFace. When $\|\widehat{\mathbf{z}_i}\| = 0$, it becomes CosFace. When

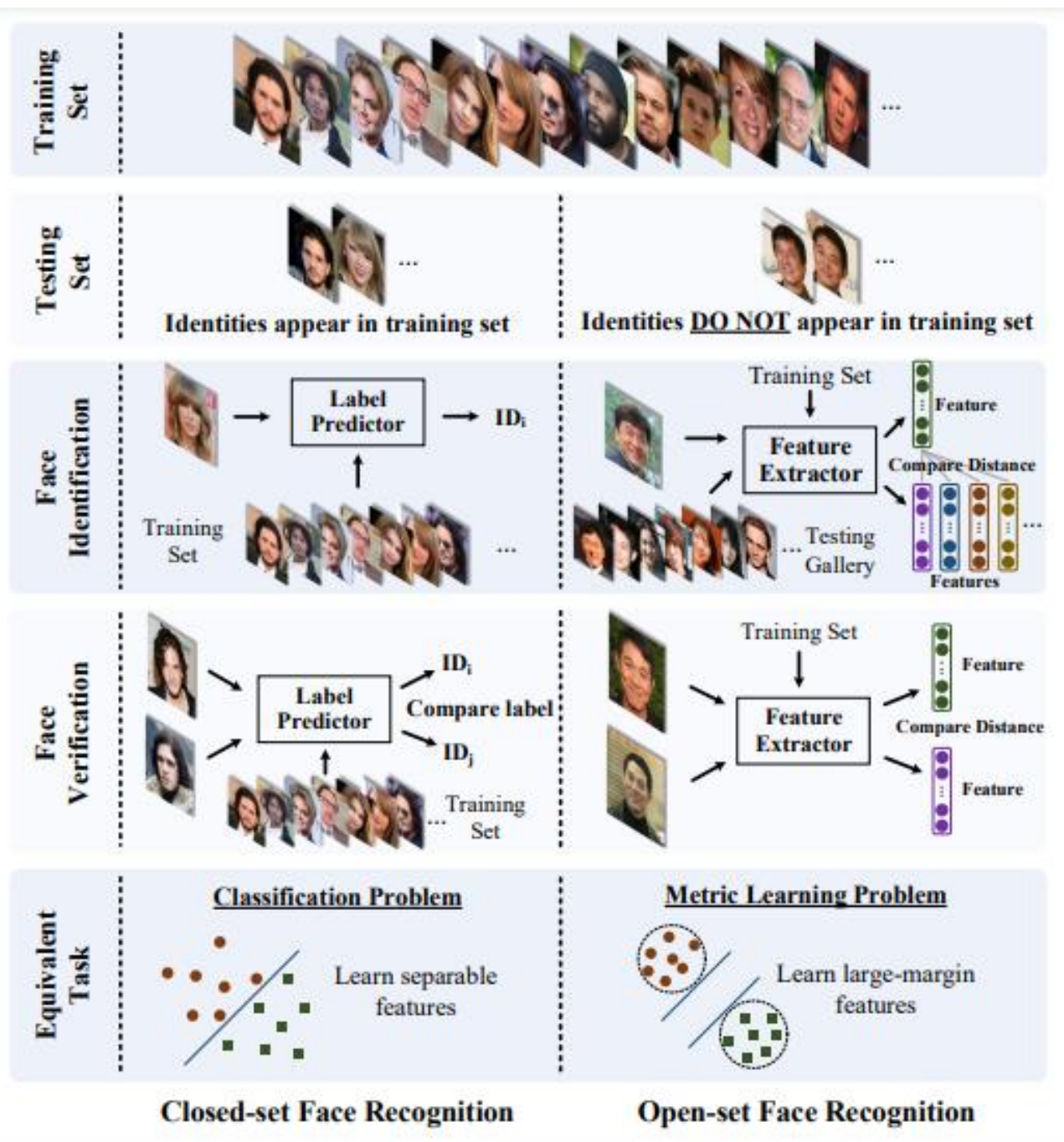
扩展：人脸识别经典数据集

- ⌚ LFW: <http://vis-www.cs.umass.edu/lfw/>
- ⌚ PubFig: <http://www.cs.columbia.edu/CAVE/databases/pubfig/>
- ⌚ YouTube Faces DB: <http://www.cs.tau.ac.il/~wolf/ytfaces/>
- ⌚ MTFL: <http://mmlab.ie.cuhk.edu.hk/projects/TCDCN.html>
- ⌚ WFLW: <https://wywu.github.io/projects/LAB/WFLW.html>
- ⌚ <https://github.com/jian667/face-dataset>
- ⌚ <https://zhuanlan.zhihu.com/p/48347016>
- ⌚ https://blog.csdn.net/weixin_44549562/article/details/124732689
- ⌚ <https://paperswithcode.com/dataset/afw>

扩展：数据集



扩展



THANKS!